

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

Duration :60 Mins
On class
Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I ARUNPRASATH R (192524077) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ARUNPRASATH R

Annexure A

ERROR ANALYSIS TASK

Q1. The following C program is intended to perform string processing, function calls, array manipulation, conditional execution, pointer operations, switch-case selection, and loop execution. However, the program contains several errors introduced during different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.

Given Program

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
    return strlen(str);
}

int main()
{
    char name[20] = "Compiler"

    int count;

    count = findLength();

    if(count > 5
    {
        printf("Length is greater than 5\n");
    }

    int marks[4] = {85, 90, 75, 80};

    printf("%d\n", marks[4]);

    int choice = 2;

    switch(choice)
    {
        case 1:
            printf("Option 1\n");
            break;

        case 2
            printf("Option 2\n");
            break;

        default:
```

```

        printf("Invalid Choice\n");
    }

    int *ptr;
    printf("%d\n", *ptr);

    int total = 100;
    int students = 0;

    float average = total / students;

    int i = 1;

    while(i <= 5)
    {
        printf("%d\n", i);
    }

    return 0;
}

```

Tasks

(a) Identify all errors in the program. Mention the line number, error type, and reason for each error. (5 Marks)

ANS:

Line 9: Syntax Error — Missing semicolon ; after the array declaration `char name[20] = "Compiler"`.

Line 11: Semantic Error — The function `findLength()` is called without passing its required `char str[]` argument.

Line 12: Syntax Error — Missing closing parenthesis `)` in the `if(count > 5)` conditional statement.

Line 17: Runtime Error — Array index out of bounds (`marks[4]` is accessed, but valid indices for an array of size 4 are 0 to 3).

Line 23: Syntax Error — Missing colon `:` after the case 2 statement.

Line 31: Runtime Error — Uninitialized pointer `ptr` is dereferenced (`*ptr`), leading to undefined behavior.

Line 35: Runtime Error — Division by zero occurs because `students` is initialized to 0 (`float average = total / students;`).

Line 39: Logical Error — The while loop lacks an increment statement for `i (i++)`, resulting in an infinite loop.

(b) Classify the identified errors as Lexical, Syntax, Semantic, Logical, or Runtime errors in a suitable table. (5 Marks)

ANS:

1. Line 9 - Missing semicolon after array initialization - Syntax Error
2. Line 11 - Function called without required arguments - Semantic Error

3. Line 12 - Missing closing parenthesis in if condition - Syntax Error
4. Line 17 - Array index out of bounds - Runtime Error
5. Line 23 - Missing colon in switch-case label (case 2) - Syntax Error
6. Line 31 - Dereferencing an uninitialized pointer - Runtime Error
7. Line 35 - Division by zero (total / students where students = 0) - Runtime Error
8. Line 39- Infinite loop due to missing loop counter increment - Logical Error

(c) Explain at which phase of compilation each category of error is detected. Also mention which errors are detected only during execution. (4 Marks)

ANS:

1. Lexical Errors: Detected during the Lexical Analysis phase when the compiler encounters character sequences that do not form valid tokens.
2. Syntax Errors: Detected during the Syntax Analysis (Parsing) phase when the token stream violates the grammatical and structural rules of the language.
3. Semantic Errors: Detected during the Semantic Analysis phase when type mismatches, undeclared identifiers, or invalid function signatures (such as incorrect arguments) are caught.
4. Logical Errors: Not detected by the compiler at all; these are flaws in the programmer's design or algorithm that cause incorrect output or behavior.
5. Runtime Errors: Errors that cannot always be anticipated during compilation and are detected only during execution when the program encounters illegal operations (such as division by zero, invalid memory access via pointers, or out-of-bound array indexing).

(d) Rewrite the program after correcting all the errors so that it executes successfully. (4 Marks)

ANS:

```
#include <stdio.h>
#include <string.h>

Int findLength(char str[])
{
    Return strlen(str);
}

Int main()
{
    Char name[20] = "Compiler"; // Added missing semicolon
    Int count;
    Count = findLength(name); // Provided the required argument
    If(count > 5) // Added missing closing parenthesis
    {
        Printf("Length is greater than 5\n");
    }
    Int marks[4] = {85, 90, 75, 80};
    Printf("%d\n", marks[3]); // Corrected out-of-bounds index from 4 to 3
    Int choice = 2;
    Switch(choice)
    {
```

```

    Case 1:
        Printf("Option 1\n");
        Break;
    Case 2: // Added missing colon
        Printf("Option 2\n");
        Break;
    Default:
        Printf("Invalid Choice\n");
}
Int val = 10;
Int *ptr = &val; // Initialized pointer to prevent undefined behavior
Printf("%d\n", *ptr);
Int total = 100;
Int students = 5; // Changed from 0 to 5 to avoid division by zero
Float average = (float)total / students;
Int I = 1;
While(I <= 5)
{
    Printf("%d\n", i);
    I++; // Added increment statement to prevent infinite loop
}
Return 0;
}

```

(e) Suggest any three compiler features or warnings that can help detect such programming errors before execution. (2 Marks)

ANS:

1. Static Analysis & Pointer Checking: Built-in compiler tools or static analyzers that track variable states to detect uninitialized pointer usage (*ptr) before the program runs.
2. Comprehensive Warning Flags (e.g., -Wall / -Wextra): Compiler flags that generate warnings for suspicious constructs, such as potential division by zero, unreachable code, or missing loop increments.
3. Strict Type and Argument Checking: Advanced semantic analysis features that rigorously check function call signatures to ensure arguments match expected declarations (catching missing arguments like in findLength()).

Q2. The following C program is intended to perform function calls, array processing, conditional execution, switch-case selection, and loop operations. However, several syntax errors have been intentionally introduced by violating the grammatical rules of the C language. These errors include missing delimiters, unmatched parentheses, incorrect control structures, malformed function definitions, and incomplete statements. Such errors are normally detected during the syntax analysis (parsing) phase of a compiler.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>
```

```

int maximum(int x, int y
{
    if(x > y)
        return x;
    else
        return y;
}

int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n")
            break;

        case 90
            printf("Maximum is 90\n");
            break;

        default:
            printf("Other Value\n");
    }

    do
    {
        printf("%d\n", max);
        max--;
    } while(max > 75);

    if(max == 75)
        printf("Limit Reached\n")
    }

    return 0;

```

Tasks

- (a) Identify all syntax errors in the program. Mention the line number and reason for each error. (4 Marks)

ANS:

- (b) Line 2: Missing closing parenthesis) in the function definition header int maximum(int x, int y.
- (c) Line 17: Missing semicolon ; at the end of the printf statement printf("Maximum is 80\n").
- (d) Line 19: Missing colon : after the case 90 statement.
- (e) Line 28: The while loop condition is missing its closing parenthesis and semicolon, written as while(max > 75) instead of being properly terminated or syntactically structured within the do-while block.
- (f) Line 31: Missing closing parenthesis) and semicolon ; for the printf statement printf("Limit Reached\n").

- (b) Classify the identified syntax errors under the appropriate categories and present them in a table. (3 Marks)

ANS:

- Line 2 - Missing closing parenthesis in function signature - Grammar Rule / Function Declaration Error
- Line 17 - Missing semicolon at the end of the statement - Delimiter Error
- Line 19 - Missing colon after the switch case label - Grammar Rule / Switch-Case Syntax Error
- Line 28 -Malformed do-while loop syntax (missing closing structure/delimiters) Control Structure - Syntax Error
- Line 31 - Unmatched parenthesis and missing semicolon in printf - Delimiter Error

- (c) Correct the syntax errors and rewrite the complete program. (4 Marks)

ANS:

```
#include <stdio.h>
```

```
Int maximum(int x, int y) // Added closing parenthesis
{
    If(x > y)
        Return x;
    Else
        Return y;
}
```

```

Int main()
{
    Int marks[5] = {70, 80, 90, 85, 95};
    Int max = maximum(marks[0], marks[1]);
    Switch(max)
    {
        Case 80:
            Printf("Maximum is 80\n"); // Added missing semicolon
            Break;
        Case 90: // Added missing colon
            Printf("Maximum is 90\n");
            Break;
        Default:
            Printf("Other Value\n");
    }
    Do
    {
        Printf("%d\n", max);
        Max--;
    } while(max > 75); // Fixed do-while syntax (added closing parenthesis and semicolon)

    If(max == 75)
        Printf("Limit Reached\n"); // Added missing closing parenthesis and semicolon

    Return 0;
}

```

(d) Explain how the parser detects syntax errors and how error recovery enables further parsing. (2 Marks)

ANS:

- **Parser Detection:** The parser checks the token stream generated by the lexical analyzer against the formal grammar rules (production rules) of the C language. When a token or sequence of tokens violates these rules (such as encountering an unexpected token or missing a mandatory delimiter like a semicolon), a syntax error is flagged.
- **Error Recovery:** To prevent the compiler from stopping at the very first error, it uses error recovery techniques (like panic-mode recovery). The parser skips tokens until it reaches a safe synchronization token (such as a semicolon or closing brace) so that it can resume parsing the rest of the code and catch subsequent errors.

(e) Suggest any two improvements that can help a compiler generate better syntax error messages. (2 Marks)

ANS:

- Context-Aware Error Diagnostics: Upgrading the parser to understand common typos so it can print specific messages (e.g., “Expected ‘:’ after case label” or “Missing semicolon before ‘}’”) rather than generic syntax failure alerts.
- Advanced Recovery Heuristics: Implementing intelligent correction suggestions that can accurately pinpoint the exact structural gap and suggest line-and-column visual indicators in modern IDEs.

Q3. The following C program is intended to perform factorial computation, function calls, array processing, pointer manipulation, conditional execution, switch-case selection, and loop operations. However, the program contains several errors introduced during different phases of compilation and execution, including lexical, syntax, semantic, intermediate code generation, code optimization, and runtime errors.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>
```

```
int factorial(int n)
```

```
{
```

```
    if(n <= 1)
```

```
        return 1;
```

```
    return n * factorial(n - 1);
```

```
}
```

```
int main()
```

```
{
```

```
    int @num = 5;
```

```
    int value = 10
```

```
    int result;
```

```
    result = factorial();
```

```
    if(result >= 100)
```

```
    {
```

```
        printf("Large Number\n");
```

```
    }
```

```

switch(result)
{
    case 24:
        printf("Factorial of 4\n");
        break;

    case 120
        printf("Factorial of 5\n");
        break;

    default:
        printf("Other Value\n");
}

int numbers[5] = {2,4,6,8,10};
printf("%d\n", numbers[5]);

int *ptr;
printf("%d\n", *ptr);

int a = 50;
int b = 0;
int c = a / b;

int sum = result + 0 + value;

int i = 1;
while(i <= 5)
{
    printf("%d\n", i);
}

return 0;
}

```

Tasks

- (a) Identify all the errors in the program. Mention the line number, error type, and reason for each error. (4 Marks)

ANS:

- Line 10: Lexical Error — Invalid character @ used in the identifier name @num.
- Line 11: Syntax Error — Missing semicolon ; at the end of the statement `int value = 10`.
- Line 13: Semantic Error — The function `factorial()` is called without passing its required `int` argument.
- Line 23: Syntax Error — Missing colon : after the case 120 statement.
- Line 28: Runtime Error — Array index out of bounds (`numbers[5]` is accessed, but valid indices for an array of size 5 are 0 to 4).
- Line 30: Runtime Error — Uninitialized pointer `ptr` is dereferenced (`*ptr`), leading to undefined behavior.
- Line 33: Runtime Error — Division by zero occurs because `b` is 0 (`int c = a / b;`).
- Line 37: Logical Error — The while loop lacks an increment statement for `I` (`i++`), resulting in an infinite loop.

(b) Classify the identified errors according to the appropriate compiler phase (Lexical, Syntax, Semantic, Intermediate Code Generation, Code Optimization, and Runtime) using a suitable table. (4 Marks)

ANS:

- Line 10- Invalid character @ in identifier - Lexical Phase
- Line 11 - Missing semicolon - Syntax Phase
- Line 13 - Function called without required argument - Semantic Phase
- Line 23 - Missing colon in switch-case label (case 120) - Syntax Phase
- Line 28 - Array index out of bounds (`numbers[5]`) - Runtime
- Line 30 - Dereferencing an uninitialized pointer - Runtime
- Line 33 - Division by zero (`a / b` where `b = 0`) - Runtime
- Line 37 - Infinite loop due to missing counter increment - Logical / Execution Behavior

(c) Explain how the compiler processes the program by identifying the errors detected during each compilation phase and those detected only at runtime. (3 Marks)

ANS:

- Lexical Phase: Scans the character stream and detects lexical errors, such as illegal tokens or symbols (e.g., the invalid @ character in @num).
- Syntax Phase: Analyzes the token structure against grammar rules and catches structural violations or missing punctuation/delimiters (e.g., missing semicolons or case colons).
- Semantic Phase: Performs type checking, scope checks, and signature validations, flagging issues like function call argument mismatches (e.g., calling `factorial()` without an integer).

- Intermediate Code Generation & Optimization Phases: Focuses on translating valid semantic structures into intermediate representations and optimizing code; while they don't typically halt execution for these specific errors, deep static checks can sometimes flag dead code or trivial arithmetic operations.
- Runtime Phase: Errors like division by zero, out-of-bound array access, and uninitialized pointer dereferencing cannot always be intercepted by static compilation rules, so they manifest and are detected only during execution when the program triggers illegal operations

(d) Rewrite the complete corrected program so that it compiles and executes without errors.
(2 Marks)

ANS:

```
#include <stdio.h>
```

```
Int factorial(int n)
```

```
{
    If(n <= 1)
        Return 1;
    Return n * factorial(n - 1);
}
```

```
Int main()
```

```
{
    Int num = 5;      // Removed invalid character '@'
    Int value = 10;   // Added missing semicolon
    Int result;
    Result = factorial(num); // Provided the required argument
    If(result >= 100)
    {
        Printf("Large Number\n");
    }
    Switch(result)
    {
        Case 24:
            Printf("Factorial of 4\n");
            Break;
        Case 120:    // Added missing colon
            Printf("Factorial of 5\n");
            Break;
        Default:
```

```

        Printf("Other Value\n");
    }
    Int numbers[5] = {2, 4, 6, 8, 10};
    Printf("%d\n", numbers[4]); // Corrected index from 5 to 4

    Int val = 10;
    Int *ptr = &val;    // Initialized pointer to prevent undefined behavior
    Printf("%d\n", *ptr);

    Int a = 50;
    Int b = 5;          // Changed from 0 to 5 to avoid division by zero
    Int c = a / b;
    Int sum = result + 0 + value;

    Int I = 1;
    While(I <= 5)
    {
        Printf("%d\n", i);
        I++;           // Added increment statement to prevent infinite loop
    }
    Return 0;
}

```

(e) Suggest any two improvements in compiler design or optimization techniques that could detect or prevent such errors before program execution. (2 Marks)

ANS:

- **Static Analysis & Bounds Checking Modules:** Integrating built-in static analyzers during compilation that evaluate array index ranges and trace pointer initialization states to catch out-of-bounds access and null/uninitialized pointer dereferences early.
- **Advanced Control Flow and Constant Propagation Optimizers:** Utilizing aggressive data-flow analysis to track variable states (such as detecting that `b` is assigned a literal 0) so the compiler can issue a compile-time warning or error for operations like division by zero.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors

Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand